Some New Results on Post Correspondence Problem and Its Modifications

Vesa Halava

Department of Mathematics, University of Turku
FIN-20014 Turku, Finland
and TUCS - Turku Centre for Computer Science
E-mail: `vehalava@utu.fi`

Tero Harju

Department of Mathematics, University of Turku
FIN-20014 Turku, Finland
E-mail: `harju@utu.fi`

Turku Centre for Computer Science
TUCS Technical Report No 388
January 2001

ISBN 952-12-0776-0
ISSN 1239-1891

Abstract

In this work we shall consider some new result on the famous word problem called the Post Correspondence Problem (PCP), initially defined by E. Post. Halava, Hirvensalo and de Wolf [4] proved that the PCP is decidable if we assume that the morphisms are marked. Moreover, Halava, Harju and Hirvensalo [2] proved that even the generalized PCP is decidable in the case of marked morphisms. As a corollary a new shorter proof was achieved for the PCP in the binary case. Here we shall consider the ideas behind these proofs. We also present a proof for the undecidability of the PCP in the case where the morphisms are permutations of each others.

Keywords: Post Correspondence Problem, generalized Post Correspondence Problem, decidability, undecidability, marked morphism, permuted morphism

TUCS Research Group

Theory Group: Mathematical Structures in Computer Science

1 Introduction

1.1 The Post Correspondence Problem

Let B be a finite alphabet. We denote by B^* the word monoid over B, where the *empty word* ε is the identity element. Moreover, let $B^+ = B^* \setminus \{\varepsilon\}$.

In the *Post Correspondence Problem* (or *PCP*, for short), we are given a set of n pairs of words,

$$\{(u_i, v_i) \mid u_i, v_i \in B^*,\ i = 1, \ldots, n\},$$

and it is asked to determine whether there exists a nonempty sequence $i_1, \ldots i_k$ of indices, where $i_j \in \{1, \ldots, n\}$ for $1 \leq j \leq k$, such that

$$u_{i_1} u_{i_2} \ldots u_{i_k} = v_{i_1} v_{i_2} \ldots v_{i_k}.$$

In this general form the PCP was proved to be undecidable by Post [7].

The PCP can be redefined by using morphisms $h, g \colon A^* \to B^*$, where $A = \{a_1, \ldots a_n\}$ is a finite alphabet of n letters and $h(a_i) = u_i$ and $g(a_i) = v_i$. Now in the PCP it is required to determine whether there exists a nonempty word $w \in A^+$ such that $h(w) = g(w)$. Such a word w is called a *solution* of the *instance* (h, g) of the PCP. The *size* of an instance is the number of letters in the domain alphabet, $n = |A|$.

The PCP offers many interesting questions related to undecidability, but there are also more concrete computational puzzles involved. One such problem is the hardness of simple instances. Small instances of the PCP, where the size of the domain alphabet and the lengths of the images of the morphisms are bounded by small numbers, have been studied by various authors, see the web page “PCP@Home”, `http://ilabws13.informatik.uni-leipzig.de/~mai97ixb/pcpcontest.html`.

Example 1. The following appealing instance was independently found by J. Waldmann and by R. J. Lorentz, see the above web page for this and other interesting instances.

Let $A = \{0, 1, 2\}$, $B = \{0, 1\}$, and define $h, g \colon A^* \to B^*$ by

$$\begin{aligned} &h(0) = 0, \quad h(1) = 1, \qquad h(2) = 011, \\ &g(0) = 1, \quad g(1) = 011, \quad g(2) = 0. \end{aligned}$$

Despite of its simple form, the length of the smallest solution to the instance (h, g) is relatively large, namely 75. This example is interesting also because in it one morphism is permuted to obtain the other one, that is, $g = h\pi$, where $\pi \colon A \to A$ is a permutation. See Theorem 1 for a general result of permuted instances. $\square$

1.2 On the edge of undecidability

The PCP is undecidable in the general case and it is one of the most important undecidable problems, since it is very useful in proving other undecidability results. For a more extensive and general treatment of undecidability, we refer to Rozenberg and Salomaa [8].

By restricting the PCP we can investigate the 'border' between decidability and undecidability. In the restricted cases some further assumptions are posed to the morphisms. For example, if we assume that the size of an instance is two, i.e., the domain alphabet A is binary, then the PCP is decidable, see Ehrenfeucht, Karhumäki and Rozenberg [1] (or [3] for a somewhat shorter proof). Note also that for the size $n = 1$ the PCP is trivially decidable. On the other hand, if $n \geq 7$, then the problem remains undecidable, see [6]. The decidability status is open for the instance sizes $3 \leq n \leq 6$. Therefore with respect to the size of the domain alphabet, the border between decidability and undecidabilitylies somewhere between three and six.

To get a feeling of the decision problems concerning morphisms, we now show that the PCP remains undecidable for the instances (h, g), where the second morphism is a permutation of the first one.

Theorem 1. *The PCP is undecidable for instances* $(h, h\pi)$, *where* $h\colon A^* \to B^*$ *is a morphism and* $\pi\colon A \to A$ *is a permutation.*

Proof. Let $h, g\colon A^* \to B^*$ be any two morphisms, where the domain alphabet is $A = \{a_0, a_1, \ldots, a_{n+1}\}$. It is well-known that it is undecidable whether such an instance (h, g) has a solution $w \in a_0(A \setminus \{a_0, a_{n+1}\})^* a_{n+1}$. (This is known as the *modified PCP*.)

Let $\overline{A} = \{\overline{a}_0, \overline{a}_1, \ldots, \overline{a}_{n+1}\}$ with $\overline{A} \cap A = \emptyset$, and let a, b, d be new letters. Define the morphisms $\mu, \eta\colon B^* \to (B \cup \{d\})^*$ by $\mu(x) = dx$ and $\eta(x) = xd$ for all $x \in B$. Now let $h'\colon (A \cup \overline{A})^* \to (B \cup \{a, b, d\})^*$ be defined by

$$\begin{aligned} h'(a_0) &= a \cdot \mu h(a_0), & h'(\overline{a}_0) &= ad \cdot \eta g(a_0), \\ h'(a_i) &= \mu h(a_i), & h'(\overline{a}_i) &= \eta g(a_i) \ \text{ for } i = 1, 2, \ldots, n, \\ h'(a_{n+1}) &= \mu h(a_{n+1}) \cdot db, & h'(\overline{a}_{n+1}) &= \eta g(a_{n+1}) \cdot b. \end{aligned}$$

Consider the permutation π of $A \cup \overline{A}$ given by $\pi(a_i) = \overline{a}_i$ and $\pi(\overline{a}_i) = a_i$. It is straightforward to show that the instance $(h', h'\pi)$ has a solution if and only if the instance (h, g) of the modified PCP has a solution. Indeed, the solutions of the instance $(h', h'\pi)$ are necessarily in the set

$$\left(a_0(A \setminus \{a_0, a_{n+1}\})^* a_{n+1} \ \cup \ \overline{a}_0(\overline{A} \setminus \{\overline{a}_0, \overline{a}_{n+1}\})^* \overline{a}_{n+1}\right)^*,$$

because of the marker letters a and b, and because of the unsynchronized morphisms μ and η. Consequently, the solutions of minimal length are either in $a_0(A \setminus \{a_0, a_{n+1}\})^* a_{n+1}$ or in $\overline{a}_0(\overline{A} \setminus \{\overline{a}_0, \overline{a}_{n+1}\})^* \overline{a}_{n+1}$.

Clearly, a word $\overline{w} \in \overline{a}_0(\overline{A} \setminus \{\overline{a}_0, \overline{a}_{n+1}\})^* \overline{a}_{n+1}$ is a solution to $(h', h'\pi)$ if and only if $w = \pi(\overline{w}) \in a_0(A \setminus \{a_0, a_{n+1}\})^* a_{n+1}$ is a solution, where w is obtained from $\overline{w}$ by removing the bars from the letters. In the case $w = a_0 v a_{n+1}$ with $v \in (A \setminus \{a_0, a_{n+1}\})^*$,

$$h'(w) = a \cdot \mu h(w) \cdot db,$$
$$h'\pi(w) = ad \cdot \eta g(w) \cdot b = a \cdot \mu g(w) db,$$

and therefore $h'(w) = h'\pi(w)$ if and only if $h(w) = g(w)$, since μ is injective. This proves the claim. $\square$

By Lecerf [5], the PCP is undecidable for injective instances, that is, for instances (h, g), where both h and g are injective morphisms. This result was improved to biprefix morphisms by Ruohonen [9]. We shall mainly consider in the following sections the case, where the morphisms h and g are marked. A morphisms is said to be *marked* if $h(x)$ and $h(y)$ start with a different letter whenever $x, y \in A$ and $x \neq y$. If the morphisms are marked then we call the instance of the PCP *marked*. Note that every marked morphism is injective. Halava, Hirvensalo and de Wolf [4] proved that the marked PCP is decidable for all alphabet sizes. We shall consider this proof in Section 2.

1.3 The generalized PCP

We can also modify the problem itself. Here we shall consider a particular modification called the *generalized* PCP (*GPCP*, for short). An instance of the GPCP consists of two morphisms $h, g \colon A^* \to B^*$ and words $p_1, p_2, s_1, s_2 \in B^*$. The problem is to determine whether or not there exists a word $w \in A^*$ such that

$$p_1 h(w) s_1 = p_2 g(w) s_2.$$

The word w is again called a *solution* of the instance $((p_1, p_2), h, g, (s_1, s_2))$ of the GPCP. It is clear that in general the GPCP is undecidable, since it is a generalization of the PCP.

In the *marked* GPCP we assume that the morphisms h and g are marked. The motivation for studying the marked PCP and the marked GPCP originates from [1], where Ehrenfeucht, Karhumäki and Rozenberg proved that the binary PCP is decidable, as we mentioned above. They actually proved that every instance of the binary PCP is either periodic or it can be reduced to an equivalent instance of the marked GPCP with binary alphabet and then showed by case analysis that the binary marked GPCP is decidable.

It was proved by Halava, Harju and Hirvensalo [2] that the marked GPCP is decidable in the general setting, that is, regardless of the size of the domain alphabet. Although the proof uses the same basic idea as [4], the technical parts involved in the generalization turned out be considerably harder than for the marked PCP. The ideas behind this proof are briefly considered in Section 3.

2 Marked PCP

2.1 Blocks

Let us first fix some notations. A word $x \in A^*$ is said to be a *prefix* of a word $y \in A^*$, if there is $z \in A^*$ such that $y = xz$. This will be denoted by $x \leq y$. Similarly, a word $x \in A^*$ is said to be a *suffix* of $y \in A^*$, if there is $z \in A^*$ such that $y = zx$. This will be denoted by $x \preccurlyeq y$. Moreover, x is a *proper* suffix of y, if $x \neq \varepsilon$ and $x \neq y$, denoted by $x \prec y$. We say that x and y are *comparable* if $x \leq y$ or $y \leq x$.

Let (h, g), where $h, g\colon A^* \to B^*$, be an instance of the marked PCP. First, we try to find the solutions of the equations

$$h(x) = g(y) \quad \text{with } a \leq h(x),\ a \leq g(y), \tag{1}$$

where the images begin with a fixed letter $a \in B$. There is an obvious method to search for a solution (if a solution exists). We define a sequence $(x_i, y_i) \in A^* \times A^*$ as follows: First, look for the letters $b, c \in A$ such that $a \leq h(b)$ and $a \leq g(c)$. These letters are unique if they exists, since the morphisms are marked. Set $(x_1, y_1) = (b, c)$. If a solution of the required type exists, then necessarily $h(x_i)$ and $g(y_i)$ are comparable, i.e., there exists a word $s \in B^*$ such that $h(x_i)s = g(y_i)$ (resp. $g(y_i)s = h(x_i)$). We shall call the word s an *overflow of* h (resp., of g).

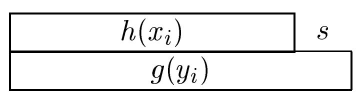

Figure 1: An overflow s of h.

If $s = \varepsilon$, then we, of course, terminate the process. Otherwise, there is at most one letter $x' \in A$ (resp. $y' \in A$) such that s and $h(x')$ (resp. $g(y')$) are comparable, since h (resp. g) is marked. Define $x_{i+1} = x_i x'$ and $y_{i+1} = y_i$ (resp. $x_{i+1} = x_i$ and $y_{i+1} = y_i y'$). We continue constructing the sequence (x_i, y_i) until we reach one of the following three cases:

(i) There is no suitable x' or y' or $h(x_i)$ and $g(y_i)$ are not comparable.
(ii) We have the same overflow s of h (resp. g) twice.
(iii) The overflow equals $s = \varepsilon$.

Since the morphisms are marked and there are only finitely many different possible overflows, we shall eventually end in the case (ii) if not the others. Since the construction of the sequence (x_i, y_i) is deterministic, the case (ii) implies that the overflows start to cycle and if we continue the process, we can never reach the cases (i) or (iii). Therefore in the case (ii), $h(x_i)$ and $g(y_i)$ are comparable for all $i \geq 1$, and the overflow is always nonempty. Clearly, in this case, there cannot exist any solutions to (1).

It is clear that if (1) has a solution, we have to terminate in the case (iii), say with (x_j, y_j). Then $h(x_j) = g(y_j)$ and there can be a solution of the instance (h, g) beginning with a only if x_j and y_j are comparable. Assume that this is the case. If $x_j = y_j$, then x_j is a solution. Otherwise, assume that $x_j = y_j z$ (resp. $y_j = x_j z$), where $d \leq z$ and $d \in A$. Then we can define $(x_{j+1}, y_{j+1}) = (x_j, y_j d)$ (resp. $(x_{j+1}, y_{j+1}) = (x_j d, y_j)$) and continue the construction of the sequence (x_i, y_i). If there is a solution of (h, g) beginning with the letter a, then we shall meet the case (iii) repeatedly until at some point $x_i = y_i$. The problem here is that we do not know when to stop if there is no solution.

Consider the sequence (x_i, y_i) up to the first case that terminates in one of the three cases. If the sequence terminates to (x_j, y_j) in the case (iii), then (x_j, y_j) is a minimal solution of the equation (1), and we call the pair (x_j, y_j) a *block* (*for the letter* a).

Assume that w is a solution of the instance (h, g). Then there exists a unique *block decomposition* of w,

$$w = u_1 u_2 \ldots u_k = v_1 v_2 \ldots v_k, \tag{2}$$

where (u_i, v_i) is a block (for some letter a_i) for all $i = 1, \ldots, k$. This means that each solution is a concatenation of blocks.

$h(w)$	$h(u_1)$	$h(u_2)$	$\cdots$	$h(u_k)$
$g(w)$	$g(v_1)$	$g(v_2)$	$\cdots$	$g(v_k)$

Figure 2: Block decomposition of a solution w

2.2 Successors

By using the blocks, we shall define for an instance (h, g) its *successor* (h', g') as follows. Let

$$A' = \{a \mid (u, v) \text{ is a block for } a \in A\}.$$

We define the morphisms $h', g' \colon (A')^* \to A^*$ by

$$h'(a) = u \quad \text{and} \quad g'(a) = v,$$

where (u, v) is a block for the letter a. Note that these new morphisms are marked, by the definition of a block, and therefore the successor (h', g') is an instance of the marked PCP. We have thus defined a new instance (h', g') of the marked PCP. We shall next prove that an instance and its successor are equivalent.

Lemma 2. *The instance $I = (h, g)$ of the marked PCP has a solution if and only if its successor $I' = (h', g')$ has a solution. Moreover, if w' is a solution of I', then $w = h'(w') = g'(w')$ is a solution of I.*

Proof. Assume first that I has a solution $w = u_1 \dots u_k = v_1 \dots v_k$, where (u_i, v_i) is block for a letter $a_i \in A'$, $1 \leq i \leq k$. By the definition of (h', g'),

$$h'(a_1 \dots a_k) = u_1 \dots u_k = w = v_1 \dots v_k = g'(a_1 \dots a_k),$$

and so $a_1 \dots a_k$ is a solution of I'.

Assume then that $w' = a_1 \dots a_k$ is a solution of I' and that, for $1 \leq i \leq k$, (u_i, v_i) is a block for a_i. Then

$$h(h'(w')) = h(u_1 \dots u_k) = g(v_1 \dots v_k) = g(g'(w')),$$

since $h(u_i) = g(v_i)$ for $1 \leq i \leq k$ by the definition of a block. This also proves the second part of the claim. $\square$

Note that, by the proof of Lemma 2, there is a one-to-one correspondence on the solutions between I and I'.

2.3 Suffix complexity

We have defined an equivalent instance I' of the instance I of the marked PCP. Next we shall prove that I' is simpler than the original instance I, and then we recursively use the construction of a successor until we obtain an instance, where the existence of a solution can be checked. We use two measures for the hardness of an instance. The first measure is the size of the domain alphabet. It is straightforward to see that $|A'| \leq |A|$, since at most $|A|$ letter can begin an image and thus be a block letter.

In the second measure, called the *suffix complexity*, we count the nontrivial suffixes of the images $h(a)$ and $g(a)$. For this, define for each morphism h,

$$\sigma(h) = | \cup_{a \in A} \{x \mid x \prec h(a)\}|,$$

and let

$$\sigma(I) = \sigma(h) + \sigma(g).$$

for the instance $I = (h, g)$. The suffix complexity was used in [1], where the following lemma was shown to hold in the binary case. In the general case, the lemma was proven in [4].

Lemma 3. *Let $I = (h, g)$ be an instance of the marked PCP and $I' = (h', g')$ be its successor. Then $\sigma(I') \leq \sigma(I)$*

Proof. For each $s \in \sigma(g')$, there exists at least one block (u, v) such that $s \preccurlyeq v$, $v' = vs^{-1}$ and for some $u' \leq u$, $h(u') = g(v')z$, where $z \in \sigma(h)$. Let $p\colon \sigma(g') \to \sigma(h)$ be the function, for which $p(s) = z$ has the minimal length in the above. Since the morphisms are marked, the word z is unique and the function p is injective. Symmetrically, we can define an injective function from $\sigma(h')$ to $\sigma(g)$, and the claim follows from these injectivities. $\square$

2.4 Decidability of the marked PCP

Next lemma is obvious.

Lemma 4. *Let ℓ be a positive integer. There exists only finitely many instances $I = (h, g)$ of the marked PCP, where $h, g\colon A^* \to B^*$ and $\sigma(I) \leq \ell$.*

Let $I_0 = (h, g)$ be an instance of the marked PCP, where $h, g\colon A_0^* \to B^*$, and let $I_i = (h_i, g_i)$ be the sequence of the successors, where $I_{i+1} = I_i'$. Assume that $h_i, g_i\colon A_i^* \to A_{i-1}^*$ for all $i \geq 1$. Note that we may assume that $A \subseteq B$, and thus $A_i \subseteq B$ for the domain alphabets for all $i \geq 0$.

Since the alphabet size and the suffix complexity do not increase, we have, by Lemma 4, one of the following three cases:

(i) $|A_j| = 1$ for some $j \geq 0$,
(ii) $\sigma(I_j) = 0$ for some $j \neq 0$,
(iii) The sequence starts to cycle, i.e., there exist n_0 and $d \geq 1$ such that, for all $j \geq n_0$, $I_j = I_{j+d}$.

The instances of the cases (i) and (ii) can be easily checked, and therefore we only need to show that the cycling case (iii) can be decided.

Consider now the cycling case. We may assume that the cycle starts already at I_0, i.e., the sequence is of the form

$$I_0, \ldots, I_{d-1}, I_d = I_0, \ldots\ . \tag{3}$$

Lemma 5. *It is decidable whether a cycling instance has a solution.*

Proof. Let $I_i = (h_i, g_i)$ be the sequence in (3). By the proof of Lemma 2, for every solution x_i of I_i, there is a solution x_{i+1} of I_{i+1} such that $x_i = h_{i+1}(x_{i+1}) = g_{i+1}(x_{i+1})$. Suppose x_0 is a solution to I_0 of minimal length. Using this reduction inductively, we obtain a solution x_d of I_d such that

$$\begin{array}{lllllllll} x_0 & = & h_1(x_1) & = & h_1h_2(x_2) & = & \ldots & = & h_1h_2\ldots h_d(x_d) \\ x_0 & = & g_1(x_1) & = & g_1g_2(x_2) & = & \ldots & = & g_1g_2\ldots g_d(x_d). \end{array}$$

Since h_i and g_i are marked, they are not length decreasing, and hence $|x_0| \geq |x_d|$. But x_0 was chosen to be a minimal length solution of I_0, and x_d is also a solution to $I_d = I_0$, and therefore $|x_0| = |x_d|$. This implies that both g_0 and h_0 map the letters occurring in x_d to letters. However, now the first letter of x_d is already a solution of I_0, and hence $|x_0| = |x_d| = 1$. Thus I_0 has a solution if and only if I_0 has a solution in A_0, which can be trivially checked. $\square$

We shall now state the whole algorithm for deciding the marked PCP.

Decision procedure for the marked PCP

(1) Set $c = 0$, $i = 0$, $I_0 = I$.

(2) Set $i = i + 1$, and construct the successor I_i.

(3) If I_i has domain alphabet of size 1 or $\sigma(I_i) = 0$, then check whether I_i has a solution, print the result and terminate.

(4) If I_i is simpler than I_{i-1} (i.e., it has a smaller domain alphabet or $\sigma(I_i) < \sigma(I_{i-1})$), set $c = i$ and goto 2.

(5) If I_i is not simpler than I_{i-1}, then search whether $I_i = I_j$ for some $c \leq j \leq i - 1$. If there is such a j, then check whether there is a 1-letter solution, print the result and terminate; else goto 2.

Actually Lemma 4 gives us a computable upper bound for the number of different instances (with a fixed alphabet size and suffix complexity), and therefore we could also use a counter to ensure that we enter a loop without doing any comparisons between the instances. The time complexity of the above algorithm is exponential. By using the counter, we obtain an algorithm with polynomial space complexity.

Theorem 6. *The marked PCP is decidable.*

2.5 Nearly marked instances

Let $k \geq 1$ be an integer. A morphism $h\colon A^* \to B^*$ is said to be k*–marked*, if for any two different letters $a, b \in A$, the longest common prefix of the images $h(a)$ and $h(b)$ has length at most $k - 1$. Hence a marked morphism is 1–marked.

The following result was proved in [4] using the undecidability of the word problem of Tzeitin's semigroup [10].

Theorem 7. *The PCP is undecidable for the 2–marked morphisms.*

Note that a 2–marked morphism need not be injective. For instance, the morphism $h(a) = a$, $h(b) = ab$, $h(c) = b$ is 2–marked, but $h(b) = ab = h(ac)$. The decidability status of the PCP is open for the *strongly* 2–marked morphisms, meaning that the morphisms are 2–marked and no image is a prefix of another image.

3 Marked generalized PCP

In this section we consider very briefly the proof of the decidability of the marked generalized PCP from [2].

3.1 Equivalent instance and end blocks

Let $I = ((p_1, p_2), h, g, (s_1, s_2))$ be an instance of the marked GPCP, where $h, g\colon A^* \to B^*$ are marked morphisms and $p_i, s_i \in B^*$ for $i = 1, 2$.

We shall first simplify the instance I by removing the prefixes p_1 and p_2 from it. For this, let $\#$ be a new letter not in $A \cup B$. We extend the morphisms by defining $h(\#) = \#p_1$ and $g(\#) = \#p_2$. The original instance is equivalent to the instance $((\varepsilon, \varepsilon), h, g, (s_1, s_2))$, where $h, g\colon (A \cup \#)^* \to (B \cup \#)^*$ are marked and the solutions are from $\#A^*$. We shall denote this new instance by $(\#, h, g, (s_1, s_2))$. It is clear that if the marked GPCP is decidable for these new instances then it is decidable in the original form. Note that we cannot reduce the *end words* (s_1, s_2) in a similar fashion, because the morphisms need to be marked. However, without restriction, we can assume that $s_1 = \varepsilon$ or $s_2 = \varepsilon$, since an existence of a solution for I implies $s_1 \preccurlyeq s_2$ or $s_2 \preccurlyeq s_1$.

A solution of the marked GPCP consists of the blocks of the instance (h, g) for the marked PCP, and the (s_1, s_2)-*end blocks* (u, v), where u and v satisfy the condition $h(u)s_1 = g(v)s_2$ and there does not exists any block (r, t) such that $r \leq u$ and $t \leq v$.

Let $w \in \#A^*$ be a solution of the instance $I = (\#, h, g, (s_1, s_2))$. As in the case of the marked PCP, there is a unique *block decomposition* of w,

$$w = u_1 u_2 \ldots u_{k+1} = v_1 v_2 \ldots v_{k+1},$$

such that (u_i, v_i) is a block for each $i = 1, 2, \ldots, k$ and (u_{k+1}, v_{k+1}) is an (s_1, s_2)-end block.

$h(w)$	$h(u_1)$	$h(u_2)$	$\cdots$	$h(u_k)$	$h(u_{k+1})s_1$
$g(w)$	$g(v_1)$	$g(v_2)$	$\cdots$	$g(v_k)$	$g(v_{k+1})$

Figure 3: Block decomposition of a solution w (for $s_2 = \varepsilon$)

Note that $k = 0$ is also possible in the block decomposition of a solution. In this case the beginning block (the block for $\#$) and the end block coincide. These solutions can be considered as an end block for $\#$.

3.2 Generalized successors

Let $I = (\#, h, g, (s_1, s_2))$ be an instance of the marked GPCP, (h', g') be the successor of (h, g) and let (u, v) be an (s_1, s_2)-end block. Then

$$I'(u, v) = (\#, h', g', (s'_1, s'_2))$$

is a successor of I with respect to (u, v), where

$$(s_1', s_2') = \begin{cases} (\varepsilon, vu^{-1}), & \text{if } u \preccurlyeq v, \\ (uv^{-1}, \varepsilon), & \text{if } v \preccurlyeq u. \end{cases}$$

Note that the successor is defined only for those end blocks for which one of the suffix conditions holds.

The proof of the next lemma is a bit more difficult than the proof of Lemma 2, see [2].

Lemma 8. *The instance $I = (\#, h, g, (s_1, s_2))$ of the marked GPCP has a solution if and only if some successor $I'(u, v) = (\#, h', g', (s_1', s_2'))$ has a solution. Moreover, if w' is a solution of $I'(u, v)$, then $w = h'(w')u = g'(w')v$ is a solution of I.*

3.3 Overview of the proof

Let $I = (\#, h, g, (s_1, s_2))$ be an instance of the marked GPCP. As for the marked PCP, we reduce I to its successors until we obtain instances, which are easy to check. The difference here is that at each reduction step there can be more that one successor, since there can be many different end blocks. We may think that the successors form a tree, where the morphisms (but not the instances) are equal for all instances at the same depth of the tree.

As in the case of the marked PCP, also here the sequence of the pairs of morphisms is eventually periodic. However, the tree of the instances (carrying the end words (s_i, s_i')) can be infinite. Indeed, although for each letter $a \in (B \cup \#)$ there exists at most one block, there might be several, even infinitely many, end blocks for a. The first important step is to prove that we can manage the infinitude of the end blocks.

It might happen that for a letter a there exists a 5-tuple (x, y, r, s, z) such that, for all $k \geq 0$, $(xr^k z, ys^k)$ (resp. $(xr^k, ys^k z)$) is an (s_1, s_2)-end block. But it can be proved that if there is such a 5-tuple (*extendible end block*), then the size of the domain alphabet will decrease, and in the next reduction step we need to consider only finitely many different successors from the end blocks $(xr^k z, ys^k)$ (resp. $(xr^k, ys^k z)$), $k \geq 0$. Moreover, if we have a new end block from an extendible end block, then the size of the alphabet decreases again. Note that there can be only finitely many extendible end blocks for a letter a.

In particular, if the morphisms cycle (as in the marked PCP), the alphabet size does not decrease, and therefore, in this case, there cannot be extendible end blocks. Since in the beginning of the cycle, there are only finitely many instances, this finiteness property holds throughout the cycle.

Since the cases for the alphabet size 1, and the cases, where the suffix complexity of the morphisms is zero, can be easily checked, we need to consider the *cycling instances*, where the morphisms enter a cycle.

The second important step in the proof is to find an upper-bound for the lengths of the new end words s in the cycle. This can be done by considering the *successor sequence of the minimal solutions*, where in every reduction step we consider the successor with respect to the end block in a solution of minimal length. It can be proven that in the successor sequence of the minimal solutions the length of the new end words can increase only up to some fixed limit. It follows that there are only finitely many possible end words to be considered, and thus in a cycle there are only finitely many instances to be checked.

Consider now the successor tree of an instance. The decision in every branch of the tree is performed as follows:

(i) If there is no end block, then the branch terminates and has no solution.

(ii) If the same instance occurs twice, then terminate the branch, since there is no solution in this branch.

(iii) If the length of an end word is more that the limit, then terminate the branch, since it is not a branch of a minimal solution.

(iv) If an end block (u, u) occurs, then determine whether the marked (not generalized) PCP has a solution beginning with the letter $\#$.

(v) If an end block $(\#u, \#u)$ occurs, then the instance has a solution.

It can be seen that after a certain number of reductions each branch of the tree falls into one of the above cases.

Theorem 9. *The marked GPCP is decidable.*

It was proved in [1] that the binary PCP is decidable if the binary GPCP is decidable for marked morphisms. Therefore we have

Corollary 10. *The binary PCP is decidable.*

4 Conclusion and Future Work

We wish to mention the following open problems concerning this research:

- Is there a polynomial-time algorithm for the marked PCP? – or even for the marked GPCP?

- It follows from [9] that the strongly 5–marked PCP is undecidable. What about the decidability status of the strongly k-marked PCP for $1 < k < 5$?

- The decidability status of the PCP with elementary morphisms [8, pp. 72–77] is still open. A morphism g is *elementary* if it cannot be written as a composition $g_2 g_1$ via a smaller alphabet. Note that the marked PCP is a subcase of the PCP for elementary morphisms.

References

[1] A. Ehrenfeucht, J. Karhumäki, and G. Rozenberg, *The (generalized) Post Correspondence Problem with lists consisting of two words is decidable*, Theoret. Comput. Sci. **21** (1982), 119–144.

[2] V. Halava, T. Harju, and M. Hirvensalo, *Generalized PCP is decidable for marked morphisms*, FCT'99 (G. Ciobanu and G. Păun, eds.), Lecture Notes in Comput. Sci., vol. 1684, Springer-Verlag, 1999, to appear in Internat. J. Algebra Comput., pp. 304–315.

[3] V. Halava, T. Harju, and M. Hirvensalo, *Binary (generalized) Post Correspondence Problem*, Tech. Report 357, Turku Centre for Computer Science, August 2000, to appear in Theoret. Comput. Sci.

[4] V. Halava, M. Hirvensalo, and R. de Wolf, *Decidability and undecidability of marked PCP*, STACS'99 (C. Meinel and S. Tison, eds.), Lecture Notes in Comput. Sci., vol. 1563, Springer-Verlag, 1999, pp. 207–216.

[5] Y. Lecerf, *Récursive insolubilité de l'équation générale de diagonalisation de deux momomorphisms de monoïdes libres*, Computes Rendus Acad. Sci. Paris **257** (1963), 2940–2943.

[6] Y. Matiyasevich and G. Sénizergues, *Decision problems for semi-Thue systems with a few rules*, Proceedings, 11^{th} Annual IEEE Symposium on Logic in Computer Science (New Brunswick, New Jersey), IEEE Computer Society Press, 27–30 July 1996, pp. 523–531.

[7] E. Post, *A variant of a recursively unsolvable problem*, Bulletin of Amer. Math. Soc. **52** (1946), 264–268.

[8] G. Rozenberg and A. Salomaa, *Cornerstones of Undecidability*, Prentice Hall, 1994.

[9] K. Ruohonen, *Reversible machines and Post's correspondence problem for biprefix morphisms*, Elektron. Informationsverarb. Kybernet. (EIK) **21** (1985), no. 12, 579–595.

[10] G. C. Tzeitin, *Associative calculus with an unsolvable equivalence problem*, Tr. Mat. Inst. Akad. Nauk (1958), no. 52, 172–189, In Russian.

Turku Centre for Computer Science
Lemminkäisenkatu 14
FIN-20520 Turku
Finland

http://www.tucs.abo.fi

University of Turku
- **Department of Mathematical Sciences**

Åbo Akademi University
- **Department of Computer Science**
- **Institute for Advanced Management Systems Research**

Turku School of Economics and Business Administration
- **Institute of Information Systems Science**